

Multi-Account Email Manager — System Design Document

Project: MailHub — Full-Stack Multi-Account Email Management System

Stack: Next.js (App Router) + Node.js + PostgreSQL + Redis

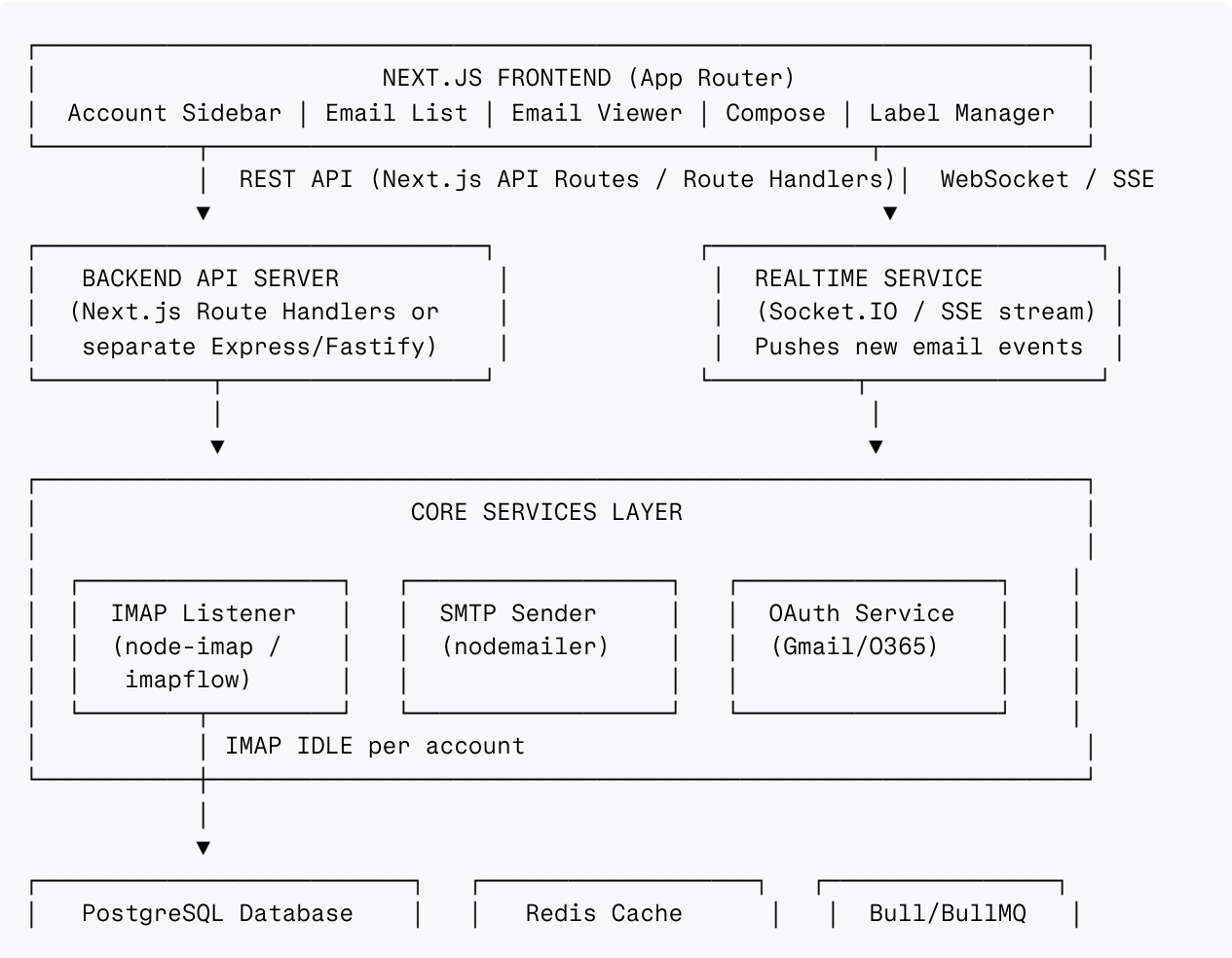
Version: 1.0

Author: System Architecture Team

1. Executive Summary

MailHub is a full-stack email management platform supporting 100+ email accounts across IMAP/SMTP and OAuth-based providers. The system continuously polls or listens for new emails via IMAP IDLE across all connected accounts, persists email data in a PostgreSQL database, and exposes a RESTful API for account management and email operations. The Next.js frontend provides a clearly account-separated UI, real-time push notifications via WebSocket/SSE, inbox/draft/sent views per account, and a label-based organization system.

2. System Architecture Overview



- accounts - emails - attachments - labels - account_labels - email_labels	- sessions - email counts - WS presence	- Job Queue - sync jobs - send jobs
--	---	---

3. Technology Stack

Layer	Technology	Rationale
Frontend	Next.js 14+ (App Router)	SSR + RSC, API routes, file-based routing
Styling	Tailwind CSS v4	Utility-first, rapid UI development
State Management	Zustand + React Query (TanStack)	Local UI state + server state sync
Real-time	<u>Socket.IO</u> or SSE (Server-Sent Events)	Push new email notifications
Backend API	Next.js Route Handlers	Co-located API within Next.js project
IMAP	imapflow (Node.js)	Modern IMAP client, IDLE support
SMTP	nodemailer	Battle-tested email sending
OAuth	@auth/core / NextAuth.js v5	Gmail OAuth2, Microsoft OAuth2
Database ORM	Prisma	Type-safe DB access, migrations
Database	PostgreSQL 16	Relational, JSONB for email metadata
Cache / Queue	Redis + BullMQ	Per-account job queues, connection pool
Auth	JWT (HTTP-only cookies)	Secure session management
Background Jobs	BullMQ workers	IMAP sync, account polling fallback

4. Database Schema

4.1 Email Accounts Table

```
CREATE TABLE email_accounts (  
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  label             VARCHAR(100) NOT NULL,           -- display name e.g. "Work - Sales"  
  email_address     VARCHAR(255) NOT NULL UNIQUE,  
  provider          VARCHAR(50) NOT NULL,           -- 'imap', 'gmail', 'outlook'  
  color             VARCHAR(7),                     -- hex color for UI differentiation  
  avatar_initials   VARCHAR(3),                     -- e.g. "WS" for Work Sales  
  is_active         BOOLEAN DEFAULT true,  
  last_synced_at   TIMESTAMP WITH TIME ZONE,  
  
  -- IMAP/SMTP credentials (encrypted at rest)  
  imap_host        VARCHAR(255),
```

```

imap_port      INTEGER,
imap_secure    BOOLEAN DEFAULT true,
smtp_host      VARCHAR(255),
smtp_port      INTEGER,
smtp_secure    BOOLEAN DEFAULT true,
username       VARCHAR(255),
password_encrypted TEXT,                -- AES-256-GCM encrypted

-- OAuth credentials
oauth_provider VARCHAR(50),            -- 'google', 'microsoft'
oauth_access_token TEXT,
oauth_refresh_token TEXT,
oauth_token_expiry TIMESTAMP WITH TIME ZONE,

created_at     TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
updated_at     TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

```

4.2 Emails Table

```

CREATE TABLE emails (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id        UUID NOT NULL REFERENCES email_accounts(id) ON DELETE CASCADE,
  message_id        VARCHAR(512) NOT NULL,                -- RFC 2822 Message-ID header
  uid               BIGINT,                                -- IMAP UID
  thread_id         VARCHAR(512),                          -- for threading
  folder            VARCHAR(100) NOT NULL,                 -- 'INBOX', 'SENT', 'DRAFTS', 'TRASH'
  subject           TEXT,
  body_html         TEXT,
  body_text         TEXT,
  from_address      VARCHAR(512),
  from_name         VARCHAR(255),
  to_addresses      JSONB,                                -- [{ name, address }]
  cc_addresses      JSONB,
  bcc_addresses     JSONB,
  reply_to          VARCHAR(512),
  in_reply_to       VARCHAR(512),
  is_read           BOOLEAN DEFAULT false,
  is_starred        BOOLEAN DEFAULT false,
  is_draft          BOOLEAN DEFAULT false,
  has_attachments   BOOLEAN DEFAULT false,
  size_bytes        INTEGER,
  received_at       TIMESTAMP WITH TIME ZONE,
  sent_at           TIMESTAMP WITH TIME ZONE,
  raw_headers       JSONB,
  created_at        TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  UNIQUE(account_id, message_id)
);

CREATE INDEX idx_emails_account_folder ON emails(account_id, folder);
CREATE INDEX idx_emails_received_at ON emails(received_at DESC);
CREATE INDEX idx_emails_thread ON emails(thread_id);

```

4.3 Attachments Table

```
CREATE TABLE attachments (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email_id    UUID NOT NULL REFERENCES emails(id) ON DELETE CASCADE,  
  filename     VARCHAR(512),  
  content_type VARCHAR(255),  
  size_bytes   INTEGER,  
  storage_path TEXT,                                -- S3/local path  
  cid          VARCHAR(255),                        -- for inline attachments  
  created_at   TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

4.4 Labels Table

```
CREATE TABLE labels (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  name        VARCHAR(100) NOT NULL,  
  color       VARCHAR(7) NOT NULL DEFAULT '#3B82F6',  
  description TEXT,  
  icon        VARCHAR(50),  
  created_at  TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  UNIQUE(user_id, name)  
);
```

4.5 Account-Label Junction Table

```
-- Assign labels to accounts (for grouping accounts by purpose)  
CREATE TABLE account_labels (  
  account_id  UUID REFERENCES email_accounts(id) ON DELETE CASCADE,  
  label_id    UUID REFERENCES labels(id) ON DELETE CASCADE,  
  PRIMARY KEY (account_id, label_id)  
);
```

4.6 Email-Label Junction Table

```
-- Tag individual emails with labels  
CREATE TABLE email_labels (  
  email_id    UUID REFERENCES emails(id) ON DELETE CASCADE,  
  label_id    UUID REFERENCES labels(id) ON DELETE CASCADE,  
  PRIMARY KEY (email_id, label_id)  
);
```

5. Backend Architecture

5.1 IMAP Connection Manager

The IMAP Connection Manager maintains a pool of persistent IMAP connections — one per active account — using `imapflow`. It leverages **IMAP IDLE** to receive server-push notifications when new mail arrives, avoiding the overhead of continuous polling.

```
IMAPConnectionManager
├─ connectionPool: Map<accountId, ImapFlow>
├─ initializeAccount(account)    → opens IMAP connection + IDLE
├─ destroyAccount(accountId)     → closes connection, removes from pool
├─ refreshOAuthToken(account)    → renews OAuth2 access_token before expiry
├─ onNewEmail(accountId, uid)    → fetches email, persists to DB, emits WS event
├─ syncFolder(accountId, folder) → full initial sync on first connect
├─ reconnect(accountId)         → exponential backoff retry on disconnect
```

Connection lifecycle:

1. On system startup, load all active accounts from DB.
2. For each account, open an `imapflow` connection (IMAP/SSL or OAuth2 XOAUTH2).
3. Enter IDLE mode on INBOX. On new email event: fetch message, parse with `mailparser`, persist to `emails` table, emit WebSocket event.
4. Rotate through folders (SENT, DRAFTS, TRASH) via scheduled BullMQ sync jobs (every 5 minutes for SENT/DRAFTS, every hour for TRASH).
5. On OAuth token near-expiry (5 minutes before), proactively refresh and update DB.

Handling 100+ connections:

- Use a **worker process** (or Node.js cluster) dedicated to IMAP connections separate from the HTTP API process.
- Cap simultaneous IDLE connections at provider limits (Gmail: 15 concurrent IMAP sessions per account; handle with session recycling).
- Use BullMQ with a rate limiter to throttle initial syncs at startup (e.g., 10 accounts/second).

5.2 SMTP Sender Service

```
// src/services/smtp-sender.ts
export async function sendEmail(accountId: string, payload: SendEmailPayload) {
  const account = await getDecryptedAccount(accountId);

  const transporter = account.provider === 'gmail'
    ? createGmailTransporter(account.oauthAccessToken)
    : createSMTPTransporter(account);

  const info = await transporter.sendMail({
    from: `"${account.label}" <${account.emailAddress}>`,
    to: payload.to,
```

```

    cc: payload.cc,
    bcc: payload.bcc,
    subject: payload.subject,
    html: payload.bodyHtml,
    text: payload.bodyText,
    attachments: payload.attachments,
    inReplyTo: payload.inReplyTo,
    references: payload.references,
  });

  // Persist sent email to DB
  await persistSentEmail(accountId, info, payload);
  return info;
}

```

5.3 OAuth2 Flow

Gmail OAuth2:

1. Frontend redirects to `/api/auth/oauth/google?accountId=new`
2. Backend exchanges code for `access_token` + `refresh_token`
3. Credentials stored encrypted in `email_accounts` table
4. IMAP connection uses XOAUTH2 SASL mechanism via `imapflow`

Microsoft OAuth2 (Outlook/Office 365):

- Same flow; uses EWS or IMAP with OAuth2 depending on tenant settings
- `client_credentials` flow for app-level; `authorization_code` for user-delegated

5.4 Credential Encryption

All passwords and OAuth tokens stored encrypted using **AES-256-GCM**:

```

import { createCipheriv, createDecipheriv, randomBytes } from 'crypto';

const ENCRYPTION_KEY = Buffer.from(process.env.ENCRYPTION_KEY!, 'hex'); // 32 bytes

export function encrypt(plaintext: string): string {
  const iv = randomBytes(12);
  const cipher = createCipheriv('aes-256-gcm', ENCRYPTION_KEY, iv);
  const encrypted = Buffer.concat([cipher.update(plaintext, 'utf8'), cipher.final()]);
  const authTag = cipher.getAuthTag();
  return `${iv.toString('hex')}:${authTag.toString('hex')}:${encrypted.toString('hex')}`;
}

```

6. RESTful API Specification

6.1 Authentication

All endpoints require a valid JWT in an HTTP-only cookie (Authorization: Bearer <token> header also accepted).

6.2 Account Management

Method	Endpoint	Description
GET	/api/accounts	List all email accounts for current user
POST	/api/accounts	Add new email account
GET	/api/accounts/:id	Get account details
PUT	/api/accounts/:id	Update account (label, color, credentials)
DELETE	/api/accounts/:id	Remove account and all synced emails
POST	/api/accounts/:id/test	Test IMAP/SMTP connectivity
POST	/api/accounts/:id/sync	Trigger manual full sync
GET	/api/accounts/:id/stats	Unread count, last sync time

POST /api/accounts — Request Body:

```
{
  "label": "Work - Sales",
  "emailAddress": "sales@company.com",
  "color": "#10B981",
  "provider": "imap",
  "imapHost": "mail.company.com",
  "imapPort": 993,
  "imapSecure": true,
  "smtpHost": "mail.company.com",
  "smtpPort": 587,
  "smtpSecure": true,
  "username": "sales@company.com",
  "password": "secret123"
}
```

6.3 Email Operations

Method	Endpoint	Description
GET	/api/accounts/:id/emails	List emails (supports ? folder=INBOX&page=1&limit=50)
GET	/api/accounts/:id/emails/:emailId	Get single email with full body

Method	Endpoint	Description
POST	/api/accounts/:id/emails/send	Send email via SMTP
POST	/api/accounts/:id/emails/draft	Save draft
PUT	/api/accounts/:id/emails/:emailId	Update email (mark read/starred)
DELETE	/api/accounts/:id/emails/:emailId	Move to trash / permanent delete
POST	/api/accounts/:id/emails/:emailId/reply	Reply to email
POST	/api/accounts/:id/emails/:emailId/forward	Forward email

GET /api/accounts/:id/emails — Query Parameters:

- **folder:** INBOX | SENT | DRAFTS | TRASH (default: INBOX)
- **page:** page number (default: 1)
- **limit:** page size (default: 50, max: 200)
- **search:** full-text search string
- **unreadOnly:** boolean
- **labelId:** filter by label UUID

6.4 Label Management

Method	Endpoint	Description
GET	/api/labels	List all labels for current user
POST	/api/labels	Create new label
PUT	/api/labels/:id	Update label (name, color)
DELETE	/api/labels/:id	Delete label (unassigns from all)
POST	/api/labels/:id/accounts	Assign label to account(s)
DELETE	/api/labels/:id/accounts/:accountId	Remove label from account
POST	/api/labels/:id/emails	Tag email(s) with label
DELETE	/api/labels/:id/emails/:emailId	Remove label from email
GET	/api/labels/:id/accounts	List accounts grouped under label
GET	/api/labels/:id/emails	List all emails tagged with label across accounts

6.5 Real-time Notifications

Server-Sent Events (SSE) endpoint:

```
GET /api/realtime/stream
```

- Authenticated via cookie session
- Client connects once; server pushes events as text/event-stream
- No WebSocket upgrade required; works through most proxies/load balancers

Event types pushed:

```
{ "event": "new_email", "data": { "accountId": "...", "emailId": "...", "subject": "..." } }
{ "event": "account_synced", "data": { "accountId": "...", "lastSyncedAt": "..." } }
{ "event": "account_error", "data": { "accountId": "...", "error": "Auth failed" } }
{ "event": "unread_count", "data": { "accountId": "...", "count": 12 } }
```

Frontend SSE client:

```
const es = new EventSource('/api/realtime/stream', { withCredentials: true });
es.addEventListener('new_email', (e) => {
  const data = JSON.parse(e.data);
  notificationStore.push(data);
  invalidateEmailQuery(data.accountId);
});
```

7. Frontend Architecture

7.1 Next.js App Router Structure

```
src/
├── app/
│   ├── (auth)/
│   │   ├── login/page.tsx
│   │   └── register/page.tsx
│   ├── (dashboard)/
│   │   ├── layout.tsx           ← Main app shell (sidebar + topbar)
│   │   ├── page.tsx            ← Redirect to first account inbox
│   │   ├── inbox/page.tsx      ← Unified inbox (all accounts)
│   │   ├── accounts/
│   │   │   ├── [accountId]/
│   │   │   │   ├── inbox/page.tsx
│   │   │   │   ├── sent/page.tsx
│   │   │   │   ├── drafts/page.tsx
│   │   │   └── trash/page.tsx
│   │   └── labels/
│   │       ├── page.tsx        ← Label manager
│   │       └── [labelId]/page.tsx ← View by label
```

```

├── api/
│   ├── accounts/route.ts
│   ├── accounts/[id]/route.ts
│   ├── accounts/[id]/emails/route.ts
│   ├── labels/route.ts
│   └── realtime/stream/route.ts
├── components/
│   ├── sidebar/
│   │   ├── AccountList.tsx          ← Colored account entries
│   │   ├── AccountItem.tsx
│   │   ├── LabelSection.tsx
│   │   └── AddAccountModal.tsx
│   ├── email/
│   │   ├── EmailList.tsx
│   │   ├── EmailRow.tsx
│   │   ├── EmailViewer.tsx
│   │   ├── ComposeModal.tsx
│   │   └── AttachmentList.tsx
│   ├── labels/
│   │   ├── LabelManager.tsx
│   │   ├── LabelPicker.tsx
│   │   └── LabelBadge.tsx
│   └── ui/                          ← Shared UI primitives
├── stores/
│   ├── accountStore.ts              ← Zustand: selected account, active folder
│   ├── notificationStore.ts         ← Zustand: new email queue
│   └── composeStore.ts              ← Zustand: compose modal state
├── hooks/
│   ├── useSSE.ts                   ← SSE connection manager
│   ├── useEmails.ts                ← React Query wrappers
│   └── useAccounts.ts
├── lib/
│   ├── imap/
│   │   ├── connection-manager.ts
│   │   ├── sync-worker.ts
│   │   └── oauth.ts
│   ├── smtp/
│   │   └── sender.ts
│   ├── db/
│   │   └── prisma.ts
│   └── crypto.ts

```

7.2 Account-Separation UI Design

Account differentiation strategy — each account is assigned a unique color and abbreviation to prevent confusion when sending:

- **Sidebar:** each account displayed with a colored left-border indicator, account color dot, email address, and unread badge
- **Email list header:** persistent colored banner showing current account name and email address
- **Compose window:** From: field prominently displays selected account with its color badge; account cannot be changed mid-compose without explicit action

- **Send confirmation:** modal always shows "Sending as: [colored badge] sales@company.com" before transmitting
- **Account switcher:** keyboard shortcut Cmd/Ctrl + K opens account quick-switcher with fuzzy search

COMPOSE EMAIL

[x]

FROM

☒ Work - Sales <sales@company.com>

[change]

TO

[]

CC

[]

SUBJ

[]

[Body area]

[SEND ▶]

7.3 Label-Grouped View

The label view groups accounts and emails by their assigned labels, providing a cross-account organizational perspective:

```

LABELS
├─ ☒ Client Projects (3 accounts, 47 unread)
│   ├── ☒ client-a@mycompany.com
│   ├── ☒ client-b@mycompany.com
│   └─ ☒ projectx@mycompany.com
├─ ☒ Internal (2 accounts, 5 unread)
│   ├── ☒ team@mycompany.com
│   └─ ☒ hr@mycompany.com
└─ ☒ Personal (1 account, 0 unread)
    └─ ☐ personal@gmail.com

```

Clicking a label opens a **merged inbox** view showing emails from all accounts under that label, with each email row clearly marked with its source account badge.

8. Security Considerations

8.1 Credential Storage

- IMAP/SMTP passwords encrypted with AES-256-GCM using a key stored in environment variables (or AWS KMS / HashiCorp Vault in production)
- OAuth tokens encrypted with same mechanism
- Passwords never logged or returned in API responses
- Database connections use TLS; password_encrypted column never exposed via SELECT *

8.2 API Security

- All API routes protected by JWT middleware
- Rate limiting: 100 requests/minute per user on email endpoints; 10/minute on account create
- Input validation using zod schemas on all request bodies
- SQL injection protection via Prisma parameterized queries
- CSRF protection via SameSite=Strict cookies

8.3 Email Content Security

- HTML emails rendered in a sandboxed `<iframe>` with `sandbox="allow-same-origin"` attribute to prevent XSS
- External images blocked by default; user can click "Show Images" per email
- Attachment downloads served via signed URLs with 1-hour expiry

8.4 OAuth2 Security

- OAuth state parameter validated to prevent CSRF on callback
- Refresh tokens stored server-side only; access tokens short-lived (1 hour)
- Token refresh handled transparently by the IMAP connection manager

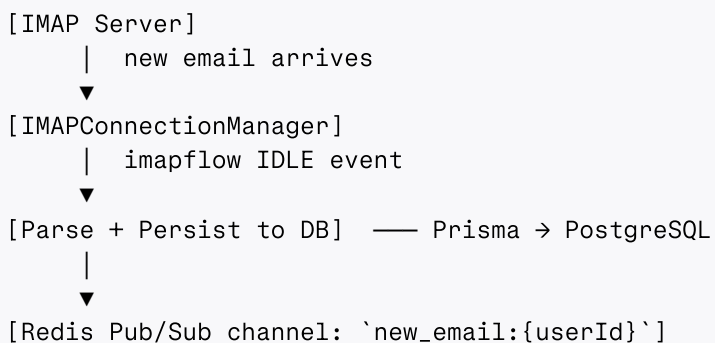
9. Real-time Architecture Detail

9.1 SSE over WebSocket Rationale

Server-Sent Events (SSE) are chosen over raw WebSocket for new email notifications because:

- SSE is a simple HTTP/1.1 compatible mechanism — no protocol upgrade complexity
- Native browser reconnect with EventSource API
- Works through most reverse proxies without special configuration
- Sufficient for unidirectional (server → client) notification flow

9.2 Event Flow





9.3 Horizontal Scaling Consideration

When deploying multiple API server instances, SSE connections are distributed across instances. Redis Pub/Sub ensures all instances receive IMAP events regardless of which instance holds the client's SSE connection.

10. Background Job Architecture (BullMQ)

10.1 Queue Definitions

Queue	Job Type	Concurrency	Description
imap-sync	sync-folder	5	Sync SENT/DRAFTS/TRASH folders
imap-sync	initial-sync	3	Full sync on account first add
email-send	send-email	10	Async SMTP send with retry
oauth-refresh	refresh-token	20	Proactive token renewal
account-health	ping-account	10	Periodic IMAP connection health check

10.2 Initial Sync Strategy

When a new account is added:

1. Immediately open IMAP connection and enter IDLE on INBOX
2. Enqueue `initial-sync` job for INBOX (last 500 emails), SENT (last 200), DRAFTS (all)
3. Job fetches email headers first (fast); bodies fetched lazily on view
4. Progress streamed back via SSE `sync_progress` events

11. Environment Configuration

```
# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/mailhub

# Redis
REDIS_URL=redis://localhost:6379
```

```

# Encryption
ENCRYPTION_KEY=64-char-hex-string    # 32 bytes for AES-256

# JWT
JWT_SECRET=your-jwt-secret
JWT_EXPIRES_IN=7d

# OAuth2 - Google
GOOGLE_CLIENT_ID=...
GOOGLE_CLIENT_SECRET=...
GOOGLE_REDIRECT_URI=https://yourdomain.com/api/auth/oauth/google/callback

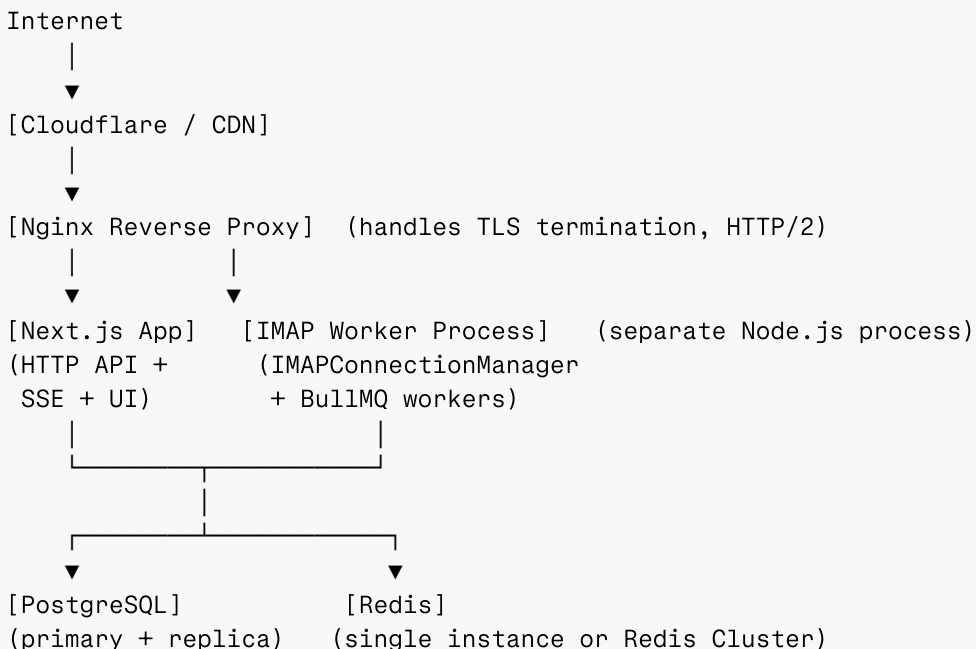
# OAuth2 - Microsoft
MICROSOFT_CLIENT_ID=...
MICROSOFT_CLIENT_SECRET=...
MICROSOFT_REDIRECT_URI=https://yourdomain.com/api/auth/oauth/microsoft/callback

# App
NEXTAUTH_URL=https://yourdomain.com
NEXTAUTH_SECRET=...

# Storage (for attachments)
S3_BUCKET=mailhub-attachments
S3_REGION=us-east-1
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...

```

12. Deployment Architecture



Process separation: The IMAP connection manager runs as a dedicated Node.js process (worker.ts) to isolate the 100+ long-lived TCP connections from HTTP request handling. Communication between API and worker happens via Redis Pub/Sub.

13. Key UI Components Specification

13.1 Account Sidebar Component

- Fixed left sidebar (280px wide on desktop, collapsible on mobile)
- Top section: user avatar, settings gear icon
- Sections: "All Inboxes" unified view, then accounts grouped by label
- Each account row: colored dot (`account.color`) + truncated email address + unread count badge
- Active account highlighted with accent background
- Add account button at bottom with + icon

13.2 Email List Component

- Account identity banner: colored strip at top with `[account.color]` background showing `account.label` and `account.emailAddress`
- Folder tabs: Inbox / Sent / Drafts / Trash
- Each email row: sender name, subject preview, time, unread dot, star toggle, label chips
- Unread emails: bold weight, accent-colored unread dot
- Multi-select checkbox on hover for bulk operations (mark read, apply label, delete)
- Infinite scroll with React Query + cursor-based pagination

13.3 Compose Modal Component

- Opens as an overlay modal (not a new page)
- **FROM field is always visible and locked to current account with colored badge**
- Quick account switch button shows account picker dropdown if user explicitly wants to change
- Auto-save draft every 30 seconds via debounced API call
- Rich text editor using `TipTap` or `react-quill`
- Attachment upload with drag-and-drop

13.4 Label Manager Page (/labels)

- Grid of label cards with name, color swatch, icon, account count, email count
- Create label form: name input, color picker (preset palette + custom hex), icon selector
- Assign accounts modal: searchable multi-select of all accounts
- Label overview: clicking label → grouped view showing all assigned accounts as expandable sections

14. Development Phases

Phase 1 — Core Infrastructure (Weeks 1–3)

- Database schema + Prisma setup
- Authentication (JWT + registration/login)
- Account CRUD API with encryption
- Basic IMAP connection manager (single account, IDLE)
- Basic SMTP sender

Phase 2 — Multi-Account Engine (Weeks 4–6)

- Scale IMAP manager to 100+ accounts with BullMQ
- OAuth2 integration (Gmail + Outlook)
- Full sync jobs (all folders)
- SSE real-time notification stream

Phase 3 — Frontend Core (Weeks 7–9)

- Next.js App Router shell with sidebar
- Account-separated email list + viewer
- Compose modal with from-account lock
- React Query integration with SSE invalidation

Phase 4 — Labels & Organization (Week 10)

- Label management API
- Account labeling + label-grouped sidebar
- Email tagging + label filter views
- Cross-account label inbox

Phase 5 — Polish & Production (Weeks 11–12)

- Full-text search (PostgreSQL `tsvector` or Meilisearch)
- Keyboard shortcuts
- Mobile responsive layout
- Deployment setup (Docker Compose)
- Performance testing at 100 accounts

15. Third-Party Library Summary

Library	Version	Purpose
imapflow	^1.0	IMAP client with IDLE support
nodemailer	^6.9	SMTP email sending
mailparser	^3.6	Parse raw MIME email messages
bullmq	^5.0	Job queue for background workers
ioredis	^5.3	Redis client (Pub/Sub + BullMQ)
@prisma/client	^5.0	PostgreSQL ORM
zod	^3.22	Request validation schemas
jose	^5.0	JWT creation and verification
tiptap	^2.0	Rich text editor for compose
zustand	^4.5	Frontend local state
@tanstack/react-query	^5.0	Server state + caching
@tanstack/react-virtual	^3.0	Virtualized email list